# Lesson 20

02.03.2025

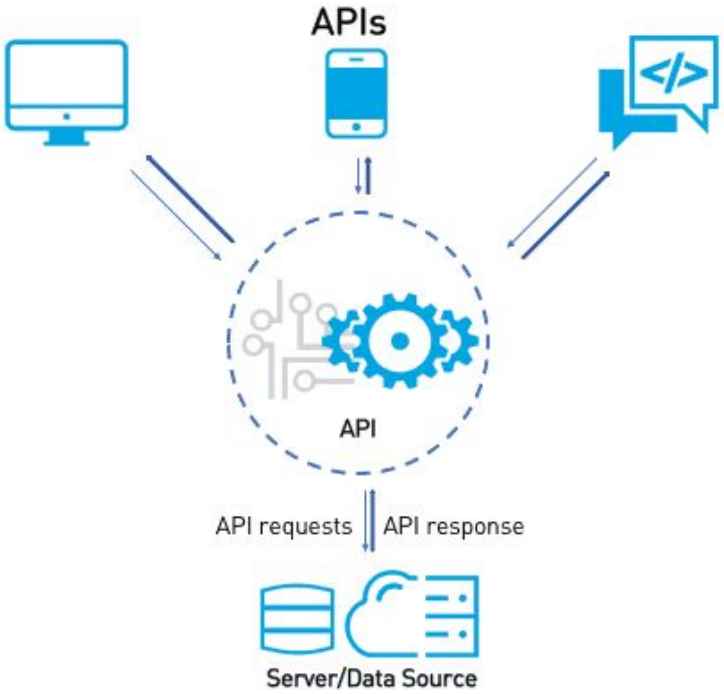
```
public class Ex1 {
    public static void main(String[] args) {
        boolean flag1 = "Java" == "Java"
                .replace( 'J' , 'J' );
        boolean flag2 = "Java" == "Java"
                .replace( 'J' , 'J' );
        System.out.println(flag1);
        System.out.println(flag2);
        System. out .println(flag1 && flag2);
    }
}
```

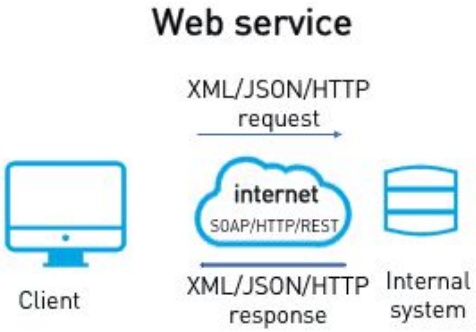
```
public class Ex2 {
    public static void main(String[] args) {
            String s1 = new String( "Java" );
            String s2 = "JaVa" ;
            String s3 = "JaVa" ;
            String s4 = "Java" ;
            String s5 = "Java" ;
            int i = 1 ;
            Integer j = 122 ;
            Integer k = 129 ;
            Integer m = 128 ;
    }
}
```

```java
public class Ex3 {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 0; i < 7; i++) {
            if (i == 4)
                break;
            else
                continue;
            sum += i;
        }
        System.out.println(sum);
    }
}
```

```
public class Ex4 {
    public static void main(String[] args) {
        String str = "BEVERAGE" ;
        String [] arr = str.split( "E" , 3 );
        System. out .println(String. join ( "." , arr));
    }
}
```

```
public class Ex5 {
    public static void main(String[] args) {
        Boolean [] arr = new Boolean[ 2 ];
        List<Boolean> list = new ArrayList<>();
        list.add(arr[ 0 ]);
        list.add(arr[ 1 ]);
        if (list.remove( 0 )) {
            list.remove( 1 );
        }
        System. out .println(list);
    }
}
```

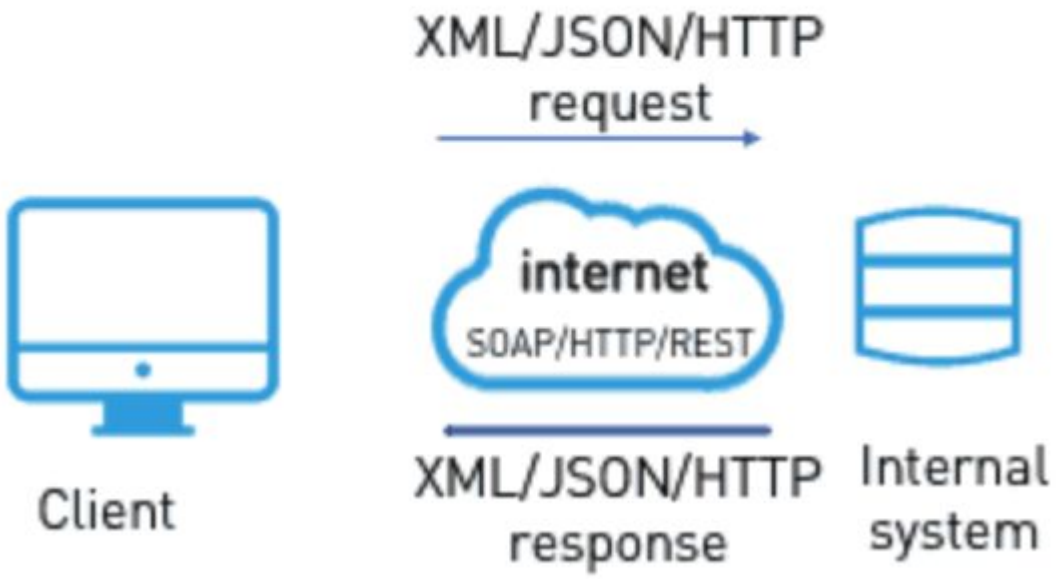


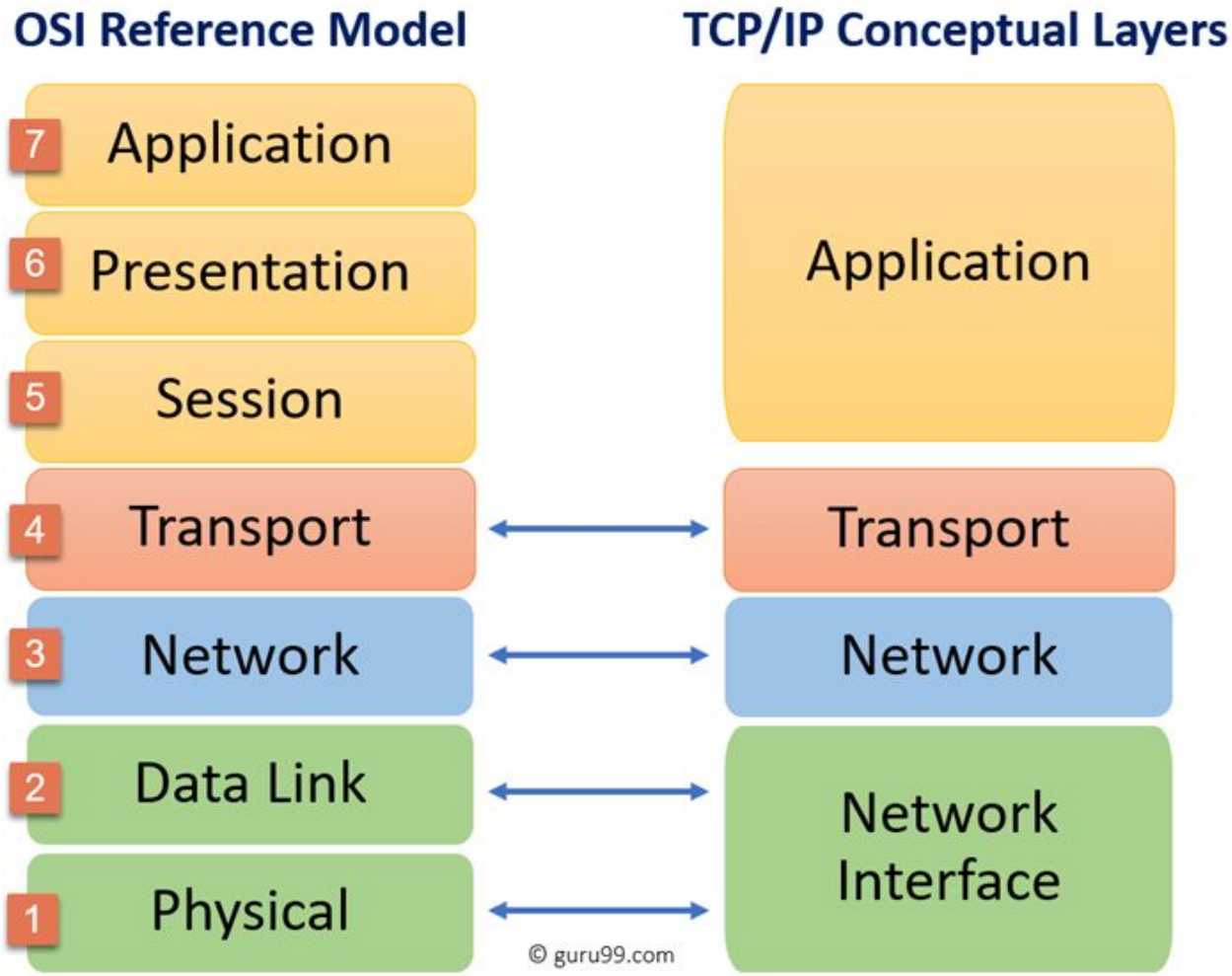



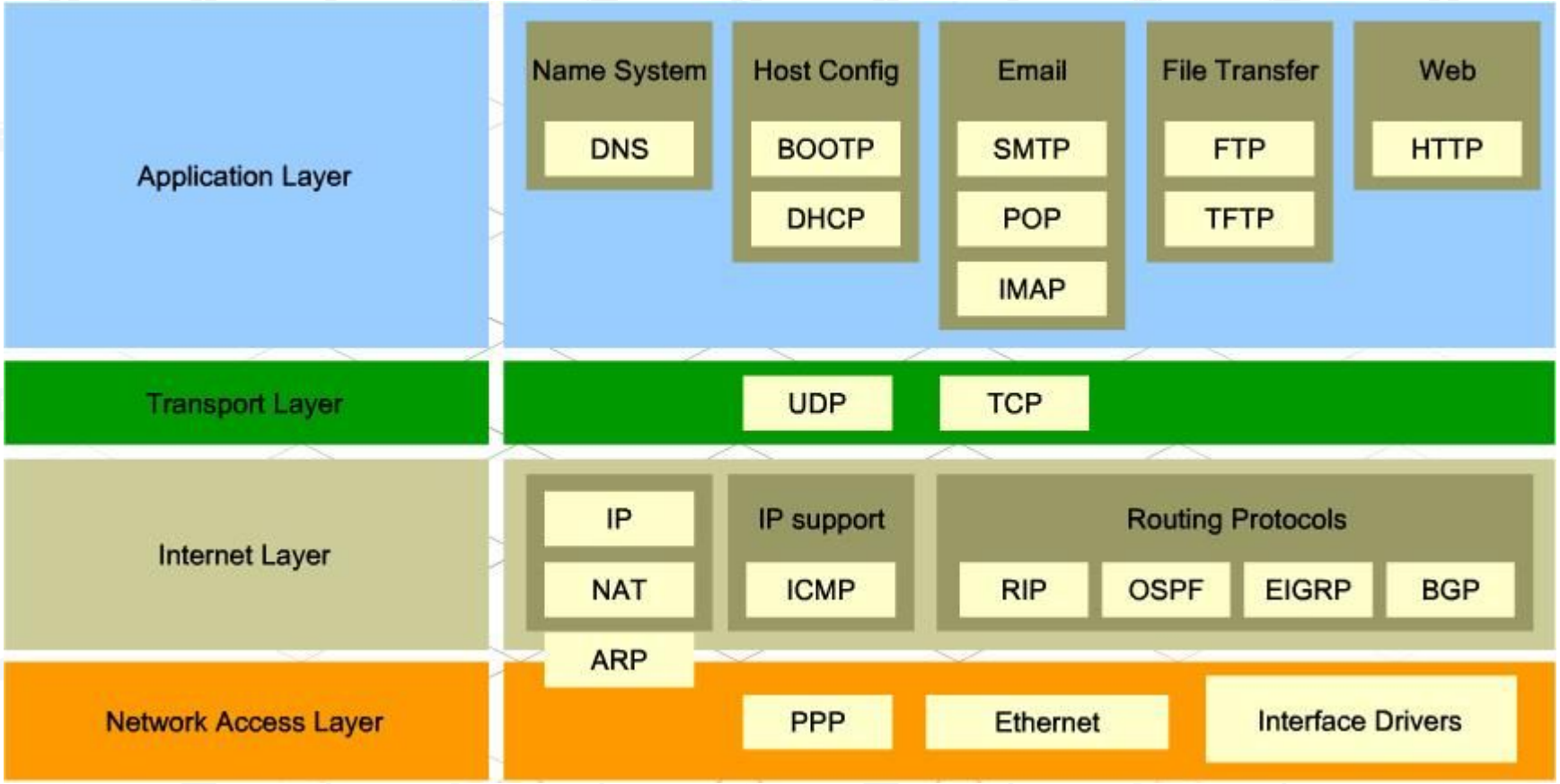

**API** - це абревіатура від інтерфейсу прикладного програмування. Це програмний інтерфейс, який дозволяє двом програмам взаємодіяти одна з одною без втручання користувача. API надає продукт або послугу для зв'язку з іншими продуктами та послугами, не знаючи, як вони реалізовані.

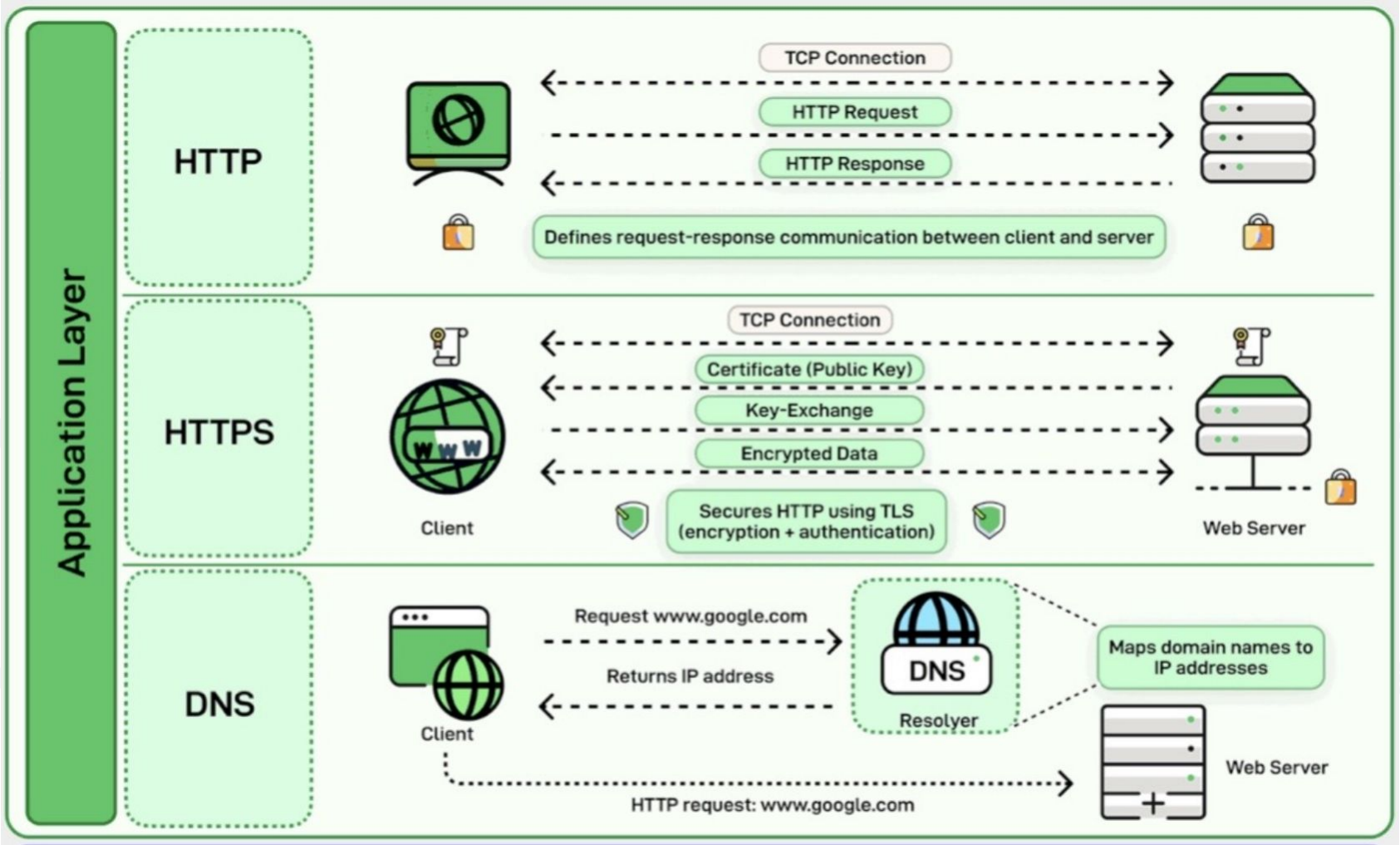
Веб-служба — це набір відкритих протоколів і стандартів, які широко використовуються для обміну даними між системами або програмами.
Програмні програми написані з використанням різних мов програмування та працюють на кількох платформах. Він дозволяє використовувати веб-сервіси для обміну даними через комп'ютерні мережі

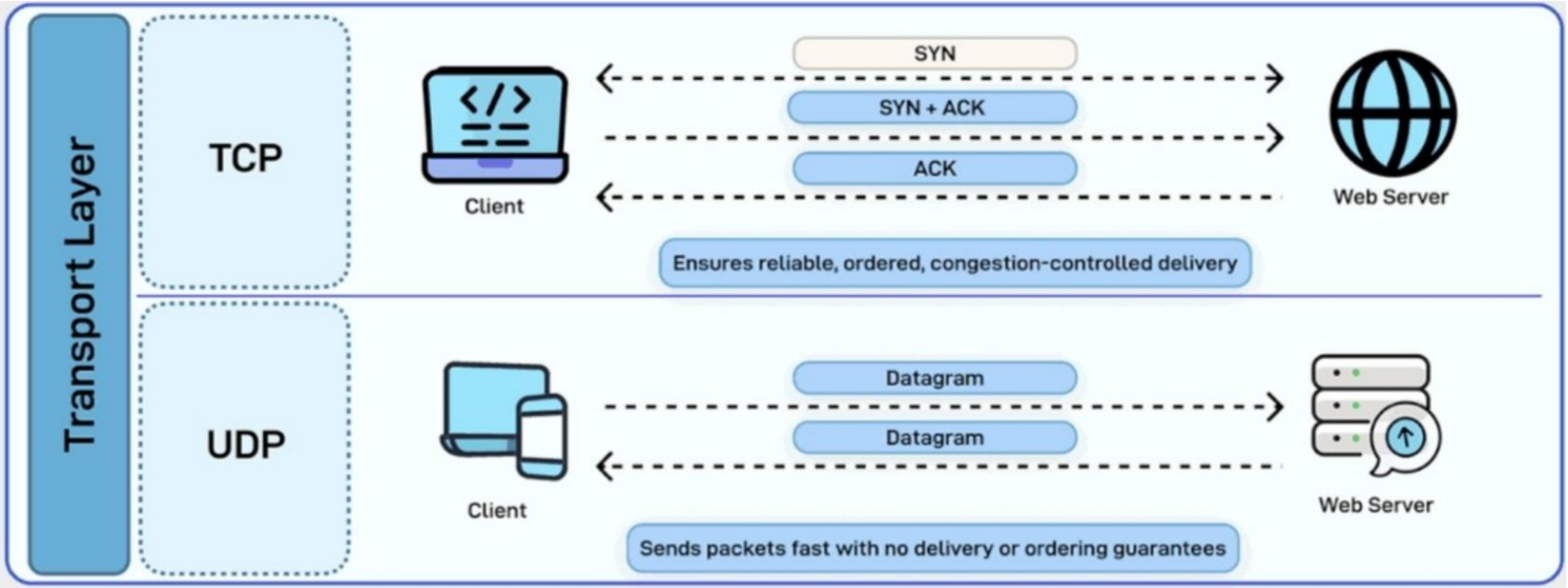
# Web service

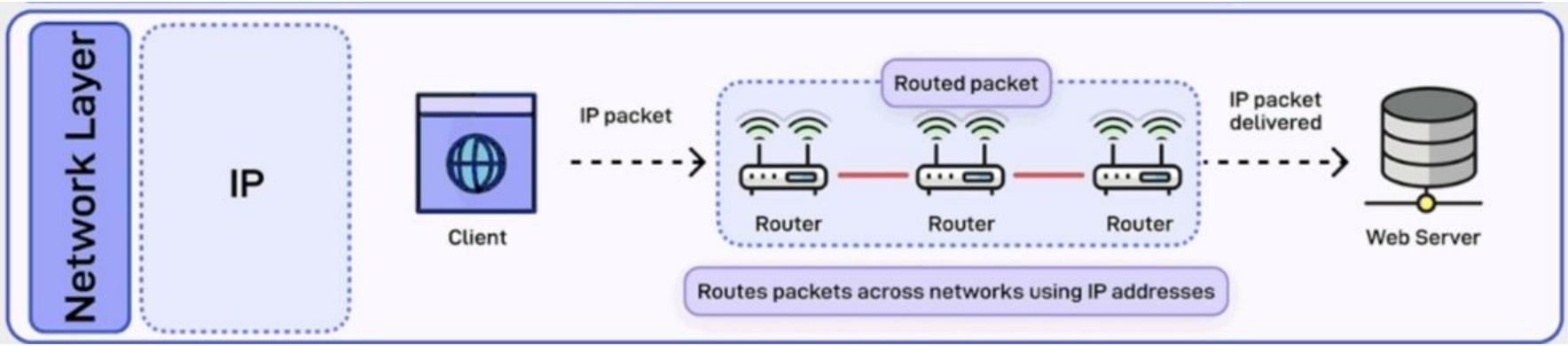

XML/JSON/HTTP
request
internet
SOAP/HTTP/REST
Client
XML/JSON/HTTP
response
Internal
system


OSI Reference Model
TCP/IP Conceptual Layers
7 Application
6 Presentation
5 Session
4 Transport
3 Network
2 Data Link
1 Physical
Application
Transport
Network
Network Interface
© guru99.com


Application Layer
Name System
DNS
Host Config
BOOTP
DHCP
Email
SMTP
POP
IMAP
File Transfer
FTP
TFTP
Web
HTTP
Transport Layer
UDP
TCP
Internet Layer
IP
NAT
ARP
IP support
ICMP
Routing Protocols
RIP
OSPF
EIGRP
BGP
Network Access Layer
PPP
Ethernet
Interface Drivers

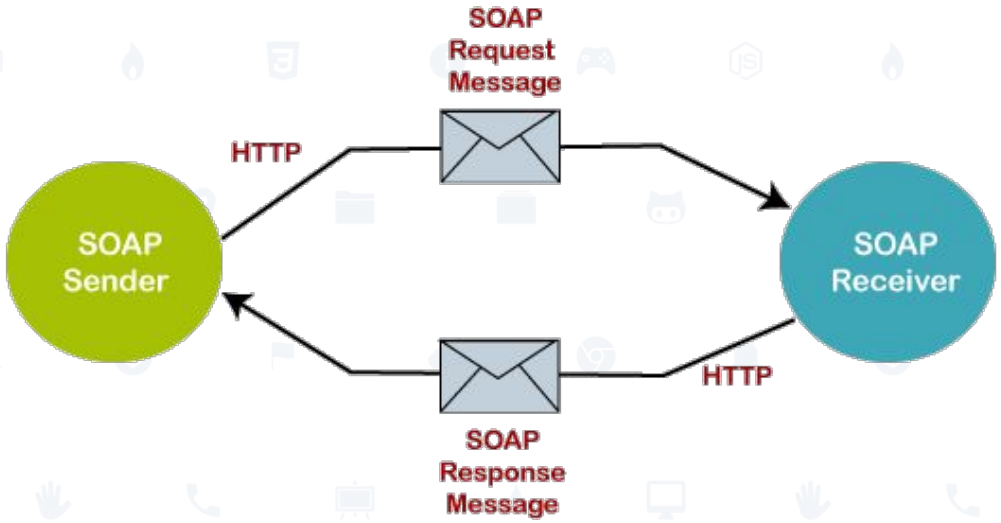

Application Layer
HTTP
TCP Connection
HTTP Request
HTTP Response
Defines request-response communication between client and server
HTTPS
TCP Connection
Certificate (Public Key)
Key-Exchange
Encrypted Data
Client
Secures HTTP using TLS
(encryption + authentication)
Web Server
DNS
Request www.google.com
Returns IP address
DNS
Resolver
Maps domain names to
IP addresses
Client
Web Server
HTTP request: www.google.com


Transport Layer
TCP
Client
SYN
SYN + ACK
ACK
Web Server
Ensures reliable, ordered, congestion-controlled delivery
UDP
Client
Datagram
Datagram
Web Server
Sends packets fast with no delivery or ordering guarantees

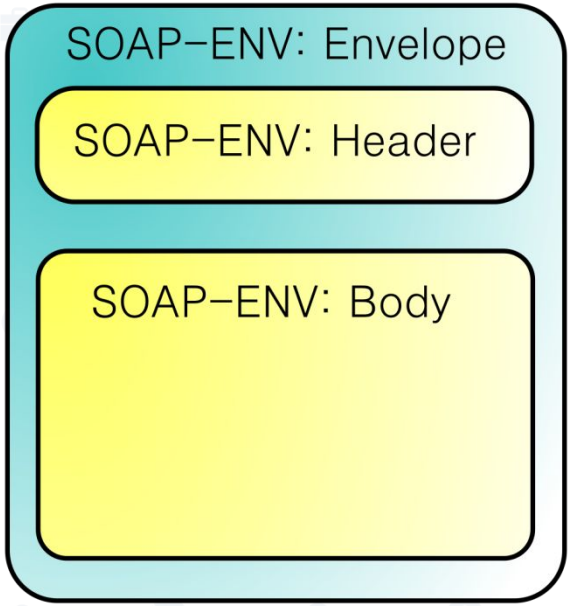

Network Layer
IP
Client
IP packet
Routed packet
Router
Router
Router
IP packet delivered
Web Server
Routes packets across networks using IP addresses

SOAP (від англ. Simple Object Access Protocol - простий протокол доступу до об'єктів) - протокол обміну структурованими повідомленнями в розподіленому обчислювальному середовищі. Спочатку SOAP призначався переважно реалізації віддаленого виклику процедур (RPC). Наразі протокол використовується для обміну довільними повідомленнями у форматі XML, а не лише для виклику процедур. Офіційна специфікація останньої версії 1.2 протоколу не розшифровує назву SOAP. SOAP є розширенням протоколу XML-RPC.

SOAP може використовуватися з будь-яким протоколом прикладного рівня: SMTP, FTP, HTTP, HTTPS та ін. Однак його взаємодія з кожним із цих протоколів має свої особливості, які мають бути визначені окремо. Найчастіше SOAP використовується поверх HTTP.



# Структура протоколу




**Envelope** – Кореневий елемент, який визначає повідомлення та простір імен, використаний у документі.

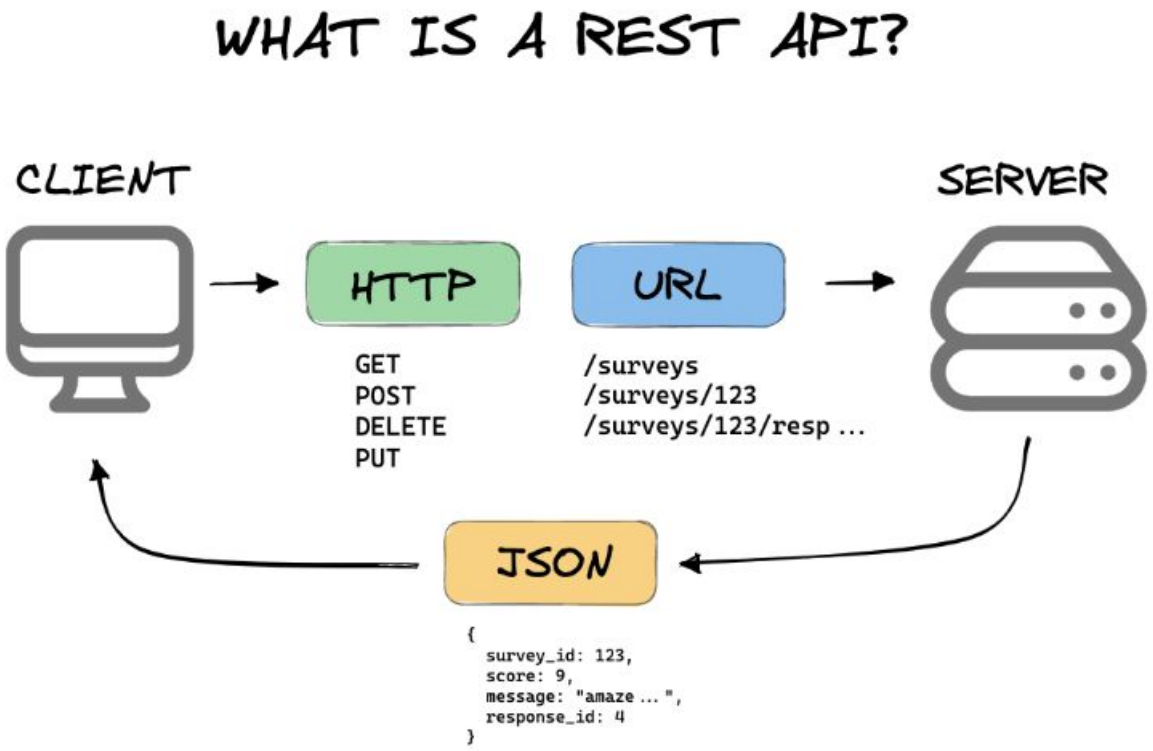
**Header** – Містить атрибути повідомлення, наприклад: інформація про безпеку або мережеву маршрутизацію.

**Body** – Містить повідомлення, яким обмінюються програми.

**Fault** – Необов'язковий елемент, який надає інформацію про помилки, що сталися під час обробки повідомлень

```xml
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
   <soap:Body>
     <getProductDetails xmlns="http://warehouse.example.com/ws">
       <productID>12345</productID>
     </getProductDetails>
   </soap:Body>
</soap:Envelope>
```

```xml
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
   <soap:Body>
     <getProductDetailsResponse xmlns="http://warehouse.example.com/ws">
       <getProductDetailsResult>
         <productID>12345</productID>
         <productName>Стакан граненый</productName>
         <description>Стакан граненый. 250 мл.</description>
         <price>9.95</price>
         <currency>
             <code>840</code>
             <alpha3>USD</alpha3>
             <sign>$</sign>
             <name>US dollar</name>
             <accuracy>2</accuracy>
         </currency>
         <inStock>true</inStock>
       </getProductDetailsResult>
     </getProductDetailsResponse>
   </soap:Body>
</soap:Envelope>
```




REST

(REpresentational State Transfer) – це архітектура, тобто. принципи побудови розподілених гіпермедіа систем, що називається World Wide Web, включаючи універсальні методи обробки та передачі станів ресурсів по HTTP

Автор ідеї та терміна Рой Філдінг 2000р.

RESTful критерії:

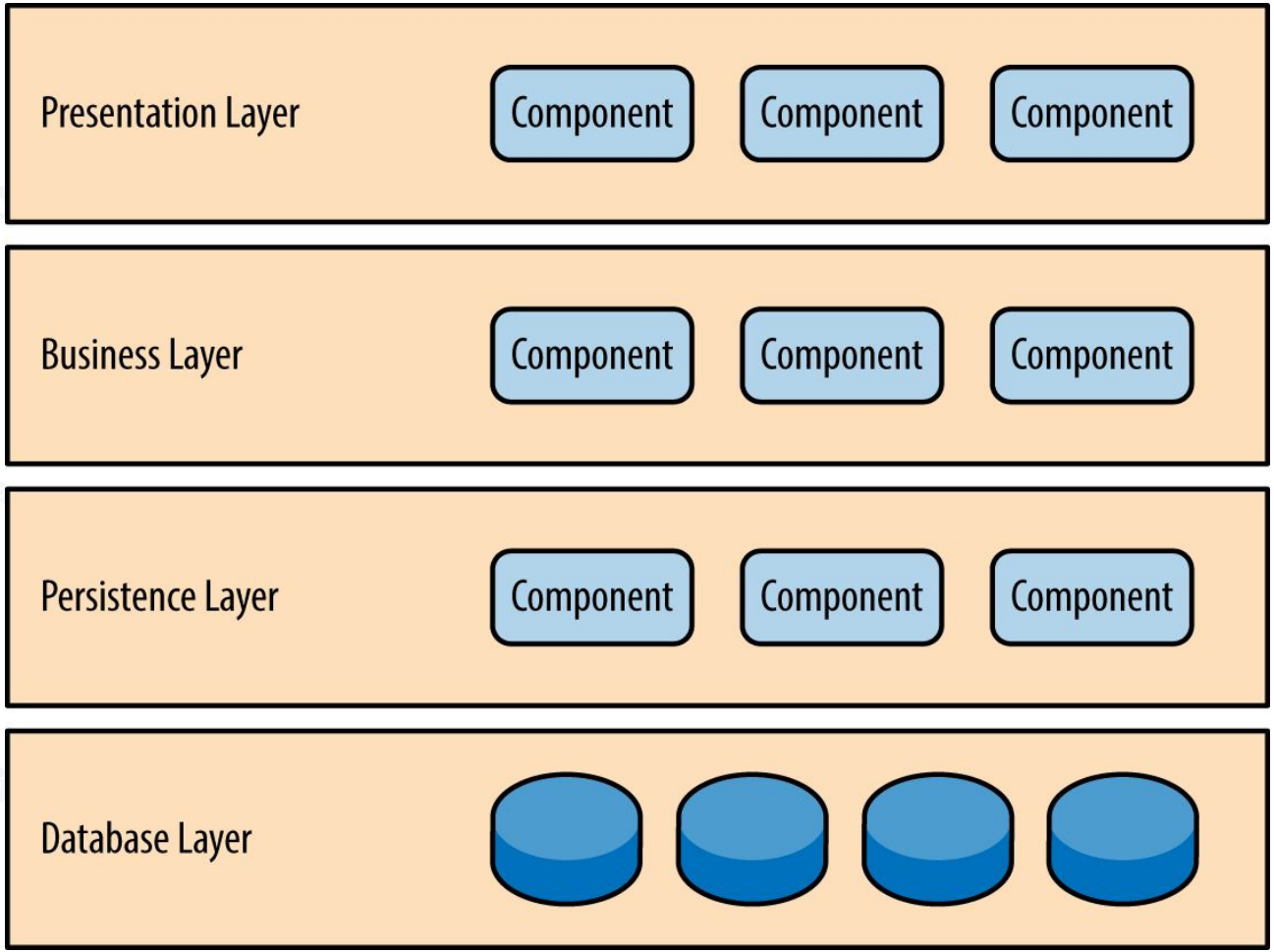
**Client-Server.** Система має бути поділена на клієнтів та на серверів. Поділ інтерфейсів означає, що, наприклад, клієнти не пов'язані зі зберіганням даних, яке залишається всередині кожного сервера, тому мобільність коду клієнта покращується. Сервери не пов'язані з інтерфейсом користувача або станом, так що сервери можуть бути простішими і масштабованішими. Сервери та клієнти можуть бути замінені та розроблятися незалежно, доки інтерфейс не змінюється.

**Stateless**. Сервер не повинен зберігати будь-яку інформацію про клієнтів. У запиті повинна зберігатись вся необхідна інформація для обробки запиту і, якщо необхідно, ідентифікації клієнта.

**Cache.** Кожна відповідь має бути позначена, чи вона кешується чи ні, для запобігання повторному використанню клієнтами застарілих або некоректних даних у відповідь на подальші запити.

**Uniform Interface.** Єдиний інтерфейс визначає інтерфейс між клієнтами та серверами. Це спрощує та відокремлює архітектуру, яка дозволяє кожній частині розвиватися самостійно.

**Layered System**. У REST допускається розділити систему на ієрархію шарів але з умовою, кожен компонент може бачити компоненти лише безпосередньо наступного шару. Наприклад:


Presentation Layer
Component
Component
Component
Business Layer
Component
Component
Component
Persistence Layer
Component
Component
Component
Database Layer

| | | |
|---|---|---|
| SAFE METHODS<br>NO ACTION ON SERVER | GET | HTTP/1.1 MUST IMPLEMENT THIS METHOD |
| | HEAD | INSPECT RESOURCE HEADERS |
| MESSAGE WITH BODY<br>SEND DATA TO SERVER | PUT | DEPOSIT DATA ON SERVER – INVERSE OF GET |
| | POST | SEND INPUT DATA FOR PROCESSING |
| | PATCH | PARTIALLY MODIFY A RESOURCE |
| | TRACE | ECHO BACK RECEIVED MESSAGE |
| | OPTIONS | SERVER CAPABILITIES |
| | DELETE | DELETE A RESOURCE – NOT GUARANTEED |

# HTTP STATUS CODES

## HTTP Status Codes

javaconceptoftheday.com

| 1xx : Informational Purpose | |
|---|---|
| 100 | Continue |
| 101 | Switching Protocols |
| 102 | Processing |
| 103 | Early Hints |

| 2xx : Success | |
|---|---|
| 200 | Ok |
| 201 | Created |
| 202 | Accepted |
| 203 | Non-Authoritative Information |
| 204 | No Content |
| 205 | Reset Content |
| 206 | Partial Content |
| 207 | Multi Status |
| 208 | Already Reported |
| 226 | IM Used |

| 3xx : Redirection | |
|---|---|
| 300 | Multiple Choices |
| 301 | Moved Permanently |
| 302 | Found |
| 303 | See Other |
| 304 | Not Modified |
| 305 | Use Proxy |
| 306 | No Longer Used |
| 307 | Temporary Redirect |
| 308 | Moved Permanently |

| 4xx : Client Errors | |
|---|---|
| 400 | Bad Request |
| 401 | Unauthorized |
| 402 | Payment Required |
| 403 | Forbidden |
| 404 | Not Found |
| 405 | Method Not Allowed |
| 406 | Not Acceptable |
| 407 | Proxy Authentication Is Required |
| 408 | Request Time Out |
| 409 | Conflict |
| 410 | Gone |
| 411 | Length Required |
| 412 | Precondition Failed |
| 413 | Payload Too Large |
| 414 | URI Too Long |
| 415 | Unsupported Media Type |
| 416 | Range Not Satisfiable |
| 417 | Expectation Failed |
| 421 | Misdirect Request |
| 422 | Unprocessable Entity |
| 423 | Locked |
| 424 | Failed Dependency |
| 425 | Too Early |
| 426 | Upgrade Required |
| 428 | Precondition Required |
| 429 | Too Many Requests |
| 431 | Request Header Fields Too Large |
| 451 | Unavailable For Legal Reasons |

| 5xx : Server Errors | |
|---|---|
| 500 | Internal Server Error |
| 501 | Not Implemented |
| 502 | Bad Gateway |
| 503 | Service Unavailable |
| 504 | Gateway Timeout |
| 505 | HTTP Version Not Supported |
| 507 | Insufficient Storage |
| 508 | Loop Detected |
| 510 | Not Extended |
| 511 | Network Authentication Required |

HTTP метод **GET** використовується **для отримання (або читання) даних ресурсу**. У разі "вдалої" (або не містить помилок) адреси, GET повертається подання ресурсу у форматі XML або JSON у поєднанні з кодом стану HTTP 200 (OK). У разі помилок зазвичай повертається код 404 (NOT FOUND) чи 400 (BAD REQUEST).

**[GET] http://www.example.com/api/v1.0/users**
(Повернути список користувачів)
**[GET]http://www.example.com/api/v1.0/users/12345**
(Повернути дані про користувача з id 12345)

HTTP метод **PUT** зазвичай використовується **для надання можливості оновлення ресурсу**. Тіло запиту при надсиланні PUT-запиту до існуючого ресурсу URI має містити оновлені дані оригінального ресурсу (повністю або тільки оновлювану частину).
При успішному оновленні за допомогою виконання PUT запиту повертається код 200 PUT не безпечна операція, так як внаслідок її виконання відбувається модифікація (або створення) екземплярів ресурсу на стороні сервера, але цей метод ідемпотентний. Іншими словами, створення або оновлення ресурсу за допомогою відправки PUT запиту - ресурс не зникне, буде розташовуватися там же, де і був при першому зверненні, а також багаторазове виконання одного і того ж PUT запиту не змінить загального стану системи

**[PUT] http://www.example.com/api/v1.0/users/12345**
(оновити дані користувача з id 12345)
**[PUT] http://www.example.com/api/v1.0/users/12345/orders/98765**
(оновити дані замовлення з id 98765 для користувача з id 12345)

HTTP метод **POST** запит найчастіше використовується **для створення нових ресурсів**. Насправді він використовується до створення вкладених ресурсів. Іншими словами, при створенні нового ресурсу, POST запит відправляється до батьківського ресурсу і, таким чином, сервіс бере на себе відповідальність на встановлення зв'язку ресурсу, що створюється з батьківським ресурсом, призначення новому ресурсу ID і т.п.

При успішному створенні ресурсу повертається HTTP код 201, а також у заголовку `Location` передається адреса створеного ресурсу.

POST не є безпечним чи ідемпотентним запитом. Тому рекомендується його використання для не ідемпотентних запитів. В результаті виконання ідентичних запитів POST надаються дуже схожі, але не ідентичні дані.

**[POST] http://www.example.com/api/v1.0/customers**
(Створити новий ресурс у розділі customers)
**[POST] http://www.example.com/api/v1.0/customers/12345/orders**
(Створити замовлення для ресурсу з id 12345)

HTTP метод **DELETE** використовується **видалення ресурсу**, ідентифікованого конкретним URI (ID).
При успішному видаленні повертається 200 (OK) код HTTP, разом із тілом відповіді, що містить дані віддаленого ресурсу. Також можливе використання коду HTTP 204 (NO CONTENT) без тіла відповіді. Якщо ви виконуєте DELETE запит на ресурс, він видаляється. Повторний DELETE запит до ресурсу закінчиться: ресурс видалено.
Проте, існує застереження щодо ідемпотентності DELETE. Повторний DELETE запит до ресурсу часто супроводжується 404 (NOT FOUND) кодом HTTP через те, що ресурс вже видалений (наприклад з бази даних) і більше не доступний. Це робить DELETE операцію не ідемпотентною, але це загальноприйнятий компроміс на той випадок, якщо ресурс був видалений з бази даних, а не помічений як віддалений.

**[DELETE] http://www.example.com/api/v1.0/customers/12345**
(Видалити з customers ресурс з id 12345)
**[DELETE] http://www.example.com/api/v1.0/customers/12345/orders/21**
(Видалити у ресурсу з id 12345 замовлення з id 21)

# Java Developer Career Path

https://roadmap.sh/java